Project Report (Group 11):

Object Detection and Classification System Using ESP32-CAM

Date: 20/12/2024

AI-Based Storage Management and Checkout System

Course: Object-Oriented Programming (OOP)

Group Members:

Nguyen Do Nguyen - 10223056

Nguyen Quoc An - 10223002

Nguyen Dinh Thai Tue - 10223071

Nguyen Xuan Son - 10222042

Table of content:

1. Introduction
2. External libraries
3. AI building
   3.1. Tools and Technologies
   3.2. Object list
   3.3. Data Acquisition and Model Training
   3.4. Deployment
4. Object-Oriented Programming
   4.1. Read Scanned Items
   4.2. Storage Management
   4.3. Checkout System
   4.4. Modify an Item’s Data
5. Conclusion
6. Future improvements
7. References

---

1. Introduction

The objective of this project is to develop an **object detection and classification system** integrated with storage management and checkout functionalities using the ESP32-CAM module. The system is divided into two major components:

- **AI Building**: Creating a machine learning model for object detection and classification using Edge Impulse. Utilizing hardware components for testing before OOP integration.
- **OOP Implementation**: Writing C++ code to manage storage (check availability, update inventory), generate lists for items before purchase and modify components' data.

This project combines artificial intelligence with object-oriented programming principles, offering a real-time solution for inventory tracking and checkout systems in an embedded environment.

---

2. External libraries

- The Program used 'termios.h' which only available on Linux-based operating systems. The program is used to establish communication between the ESP-Cam module and program through the USB port.

- An external library for object recognition is built from Edge-Impulse. The library is then uploaded with a control program to the ESP-CAM for data transmission.

---

3. AI Building

3.1 Tools and Technologies

- **Hardware**: ESP32-CAM, USB to terminal module (Arduino Uno used)
- **Software**:
    - **Edge Impulse**: For data acquisition, training, and deployment of the object detection model.
    - **Arduino IDE**: To integrate the model with ESP32-CAM.
- **Model**: FOMO (Feature Map Object Detection) based on MobileNetV2 architecture.
- **Compiler**: EON™ Compiler for optimized edge-based performance.

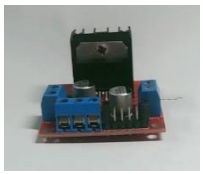
3.2 Object List

The system is designed to detect and classify the following items:

1.ESP32

2. Arduino Nano

3. Motor Driver L298N



4. Raspberry Zero

5. Raspberry Pi 3

6. Motor V1

7. 9V Battery

3.3 Data Acquisition and Model Training

- **Data Collection**: Images of each item were captured using the ESP32-CAM module and uploaded to Edge Impulse for labeling and training.
- **Training Process**:
    - The FOMO object detection model (MobileNetV2) was used for training due to its lightweight nature, making it ideal for edge devices like the ESP32-CAM.
    - Edge Impulse's **EON™ Compiler** was utilized to generate optimized Arduino libraries for model deployment.
- **Training Limitations**:
    - Due to limited processing time (20 minutes), the dataset size was kept minimal to meet the training requirements.
    - Items such as ESP32, Arduino Nano, and Motor Driver L298N exhibit visually similar features since they are primarily microcontroller-based PCBs. This can lead to occasional inaccuracies in detection and classification. (Figure 1)

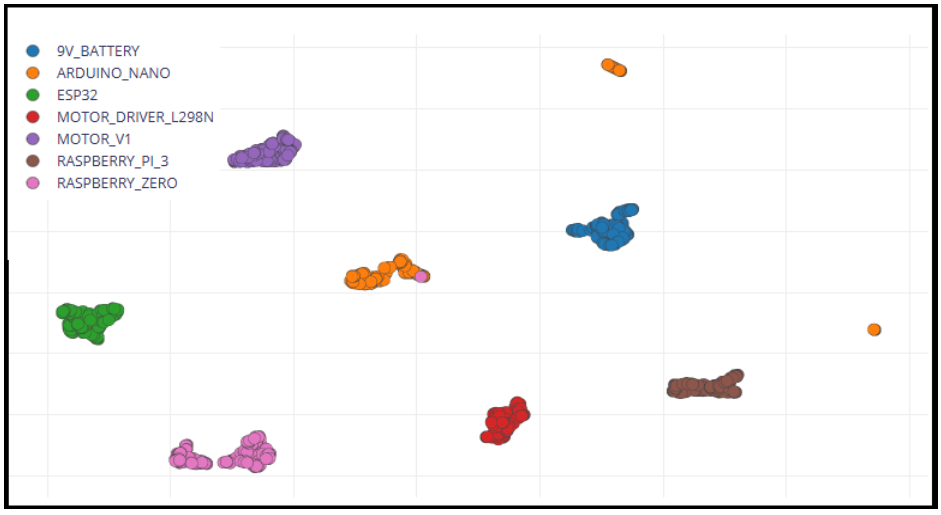


Figure 1: Item's features generated graph

3.4 Deployment

The trained model was exported as an **Arduino library** and deployed on the ESP32-CAM module. The ESP32-CAM captures real-time images, processes them using the model, and outputs the detected items.

Set-up for hardware components:

Component lists:

Arduino UNO x1

ESP32-CAM x1

Jumper cable x6

Connecting pin:

UNO	CAM
TXD – 1	IO1
RXD – 0	IO3
5V	5V
GND	GND

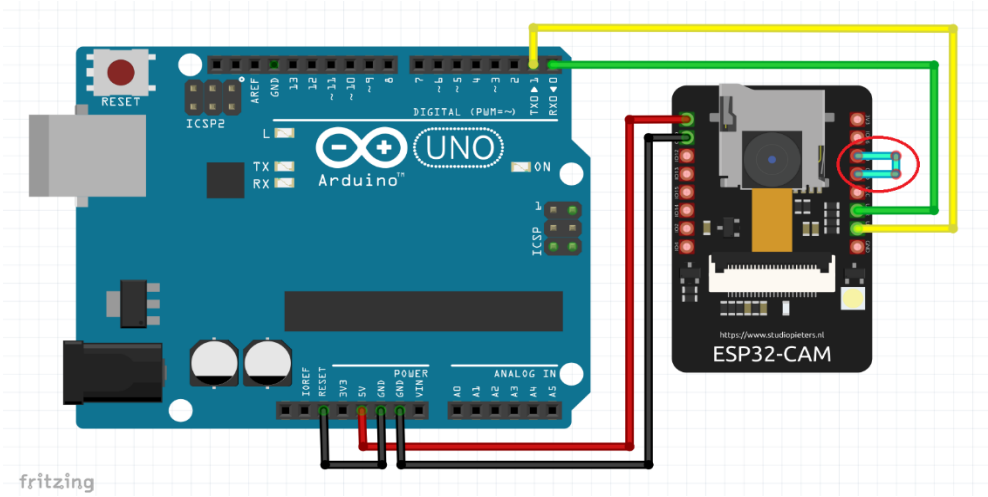


Figure 2: System wiring

In ARDUINO UNO, RESET is connected to its own GND.

Loading Arduino code from our trained AI model into ESP32-CAM:

In ESP32-CAM, IO0 is connected to its own GND to switch it into “Uploading code” mode. This is then removed after code is compiled successfully and prepared for scanning test before implementing it into OOP section.

---

4. Object-Oriented Programming

The OOP component processes the output from the AI system and introduces three key features:

4.1 Read Scanned Items

This function inside the program receives data from the scanner in the form of a string. Then compare said string to an existing database to acquire the remaining information (description, pricing, location), assigning the values to a ‘Product’ variable, which would be used throughout the program to complement other functions.

C++ Implementation:

A string is returned from the getComp() function. A constructor then opens the files ‘Desc.txt’, ‘Price.txt’, ‘Location.txt’ where the items description, price and location data are stored respectively. The matched result is then picked out and assigned.

4.2 Storage Management

The storage management system allows users to:

1. **View stored items' locations via a comprehensive map:**

   The system outputs a storage map layout in the form of a rectangular table. The rows represent the warehouses (indexed using capitalized letters A-Z) while the columns represent the storage slots (indexed using numbers). Items are designated using their first three letters with blank slots displayed as '-'. A list containing the abbreviations and respective full names is also included.

2. **Add new / Move existing items:**

   This function first asks the user to enter the component name. It then searches the storage for items with a similar name. If a match is found, the item is transferred to an input location. Otherwise, create a new entry and add it to the storage at said location. A check function is also implemented to check the vacancy of the new location. It returns an error message when the slot is already occupied and prompt the user to choose for another suitable location

   C++ Implementation:

   Use a 2-dimensional vector of class 'Product' to store the map of warehouses and its storage slots. The content is read from the file 'Location.txt' onto the vector. The vector can then be displayed or altered at will. Finally, the vector is written back to the file to save any changes.

4.3 Checkout system

The checkout system allows the user to add scanned items to a basket, viewing previously added items delete items in the basket and print out a bill with a total price when the user finished their shopping.

C++ Implementation:

Use a vector of class "Product" to store items. From the name received from the scanner, the program assigned the respective class and value from an existing file. The name of the product is then appended into a vector of strings and its price added to a total price variable. If an item from the vector is deleted, its price gets factored out of the sum.

4.4 Modify an Item's Data.

This function in the program first uses the scan function to read the item in front of the scanner. The user can then assign new value to the item, which is then stored for future usage. Fields available for modification include description and price.

C++ Implementation:

With a name string received from a scanner. The information files ('desc.txt' and 'Price.txt') is scanned to store the information in an array. The array element with the same name value as received is then modified accordingly. The array is then rewritten in the respective files.

---

5. Conclusion

This project successfully integrates object detection and classification using the ESP32-CAM with object-oriented programming to manage storage and checkout systems. By leveraging Edge Impulse for AI model development and deploying it on the ESP32-CAM, we demonstrated the feasibility of a real-time, edge-based inventory management / logistic solution.

While the system performs well under given constraints, certain inaccuracies may arise due to the limited dataset and similar visual characteristics of the items.

---

6. Future Improvements

While the project exhibits immense potential, the limitations in hardware equipment and knowledge have greatly impeded us in creating an applicable identification system.

- Many improvements would be implemented in the future to ensure a thorough system. Such a system would stop relying on the user to make the final decision of the scanning process. Instead, the scanning / reading function could be fully automated. One possible solution is to shift to local AI training instead of a web-based service could potentially bypass the limitation on training data.
- A user system is also planned in the development phase that would include the personification of function, the server/client specific functionality. With our intention to support the engineering field, our group plan to implement more OOP functions to meet their needs and organize their work better.

The time required for such features, however, has proven to be insufficient aligning to the intended timeframe. Therefore, we envisioned an expansion of features and reliability, as well as the construction of a more compact detection hardware system.

---

7. References

1. Edge Impulse Documentation
2. ESP32-CAM Datasheet
3. Arduino IDE Guide